

• We know for all indices K , $1 \leq K \leq \text{numTerms}$, $a[K].\text{col} = c$ holds for K , no. of times. By the fact that the rows of $a[]$ are in ascending order, it must be so that $a[K].\text{row} < a[K+1].\text{row}$. $\therefore$ By defn. of transposed matrix, ~~and~~ we can easily conclude that for all terms of $b[]$ having row c , $b[j].\text{col} = a[K].\text{row}$ is the largest of all such columns till now.

• By the loop invariant and the justification given above, we conclude:
For all indices K where $1 \leq K \leq \text{current_b}$, the entry $\langle a[K].\text{row}, a[K].\text{col}, a[K].\text{value} \rangle$ has been correctly transposed to the corresponding unique index K' in $b[]$, $1 \leq K' \leq \text{current_b}$ where $b[K'].\text{row} = a[K].\text{col}$, $b[K'].\text{col} = a[K].\text{row}$ and $b[K'].\text{value} = a[K].\text{value}$. The entries in $b[]$ are such that for all distinct filled indices t_1 and t_2 , $1 \leq t_1 < t_2 \leq \text{current_b}$, $b[t_1].\text{row} < b[t_2].\text{row}$, i.e. the row indices are in ascending order. Also, if $b[t_1].\text{row} = b[t_2].\text{row}$, then $b[t_1].\text{col} < b[t_2].\text{col}$, i.e. the elements of the same row are arranged in ascending order of column index. current_b denotes the highest index filled in $b[]$ so far.

• The i loop variable updates to $(i+1)$ and it's easy to see that the loop invariant holds.

Termination: The loop terminates when $i = \text{numTerms} + 1$. By the loop invariant, we conclude that the matrix $b[]$ correctly stores the transpose of matrix $a[]$. The rows in $b[]$ are in ascending order and for elements of same row, they are arranged in ascending order of column index. $\square$

Analysis: The four loops determine the computing time of fast Transpose. The bodies of the loops are executed numCols , numTerms , $\text{numCols}-1$